

Universidade Federal da Paraíba (UFPB)

Alunos: (Arthur Vieira - 20210094400) e (André Lopes - 20210066881)

Tópicos Avançados em Ciência da Computação XI

Relatório - Tetris com agentes

Experiência prévia

Arthur: Minha experiência prévia usando HTML/CSS/JavaScript é média, pois já uso para desenvolvimento web, porém nunca desenvolvi nenhum jogo usando essas linguagens.

André: Possuo experiência prévia com HTML/CSS, Porém nunca tinha usado JavaScript antes. Já desenvolvi jogos simples em java e em c#, como batalha naval e pong.

Relato de desenvolvimento

O agente escolhido foi o Claude Code da Anthropic, com o modelo sonnet 4.6. Não usamos nenhum plugin. A IDE escolhida foi o VScode.

Etapa 1

Prompt1:

Em @etapa1 crie um jogo de tetris usando HTML, CSS e JavaScript, não use nada além dessas linguagens.

O jogo deve ser simples, mas que permita a adição de várias funcionalidades em cima da base do jogo.

Dentro da pasta que mencionei crie um .md chamado descricao.md, onde deve estar descrito do que se trata o código contido na pasta e como ele funciona.

OBS.: O agente gerou uma condição de game over insuficiente com:

```
if (!isValidPosition(s.board, s.current.matrix, s.current.row, s.current.col)) {  
    s.over = true;  
}
```

De acordo com essa condição o jogo só termina se a peça nova, ao nascer, já colidir com blocos existentes ou sair dos limites permitidos. Porém o que estava acontecendo era que a peça nascia acima da tela e portanto não caia e nem colidia com os blocos existentes, o que fazia com que o jogo nunca resultasse num game over.

Prompt2:

Leia o código e o arquivo descricao.md contidos em @etapa1.

Adicione agora, em @etapa1, a feature de perder após atingir a barra superior do jogo.

Análise do Código, Fim da etapa 1

Antes da análise do código, vale destacar que, mesmo sem orientação, o agente gerou uma implementação inicial contendo mais funcionalidades do que as solicitadas, incluindo sistema de pontuação e mecânica de níveis, praticamente concluindo a segunda etapa de uma só vez.

Ao analisar o código, percebe-se que ele apresenta boa legibilidade, com funções separadas e nomes claros e bem definidos. Além disso, todos os valores numéricos foram transformados em constantes e organizados no início do código, o que favorece sua manutenção e compreensão. Entretanto, um problema relevante identificado foi a ausência de classes ou módulos. Todas as funções foram escritas em um único arquivo e diretamente no corpo principal do JavaScript, dificultando a expansão futura e a sustentabilidade do projeto. Também foram implementados poucos tratamentos de exceção, tornando o código potencialmente suscetível a diferentes tipos de bugs. Por fim, o agente adicionou poucos comentários, todos bastante breves e pouco explicativos.

Etapa 2

Prompt1:

Leia o código e o arquivo descricao.md contidos em @etapa1.

Agora, em @etapa2, use o mesmo código, porém agora adicione a feature onde as peças começam a cair mais rápido a cada nível. Também adicione a funcionalidade onde mostra o score mais alto obtido até o momento.

Em @etapa2 adicione um arquivo descricao.md que segue o mesmo estilo do que existe em @etapa1, porém no fim do arquivo detalhe o que foi adicionado, excluído e modificado.

Análise do Código, Fim da etapa 2

Grande parte do que precisava ser realizado na etapa 2 já havia sido implementada na etapa 1 pelo próprio agente, sem necessidade de orientação, como o sistema de pontuação e a mecânica de níveis. Além das funcionalidades já desenvolvidas anteriormente, foi criada a função responsável por armazenar a pontuação máxima localmente e adicionada uma animação simples de avanço de nível. Destaca-se que essa animação foi implementada pelo agente sem qualquer sugestão prévia.

Etapa 3

Prompt1:

Leia o código e o arquivo descricao.md contidos em @etapa2.

Agora, em @etapa3, use o mesmo código, porém agora adicione uma tela de abertura para esperar o usuário iniciar. Alguns jogos, ao passarem muito tempo na tela de abertura, mudam para o chamado attract mode, mostrando uma demonstração do jogo. O attract mode deve terminar se o usuário interagir de alguma forma.

Em @etapa3 adicione um arquivo descricao.md que segue o mesmo estilo do que existe em @etapa2, porém no fim do arquivo detalhe o que foi adicionado, excluído e modificado.

OBS.: O código gerado pelo agente não retornava o usuário para a tela inicial após o game over.

Prompt2:

Leia o código e o arquivo descricao.md contidos em @etapa3.

Adicione a feature em @etapa3 onde, ao dar game over, o usuário é direcionado a tela de início do jogo.

No arquivo descricao.md descreva as mudanças feitas.

Análise do Código, Fim da etapa 3

Com a adição de novas funcionalidades, percebe-se que o código está se tornando cada vez mais desorganizado. O game loop, por exemplo, apresenta grande repetição lógica entre os modos *attract* e *playing*, algo que poderia ser evitado com o uso de classes e objetos. Além disso, o game loop concentra diversas responsabilidades que poderiam ser delegadas a funções específicas, tornando o código mais simples e compreensível.

Adicionalmente, o agente implementou um algoritmo de *brute force* para o modo *attract*, no qual todas as possibilidades são testadas para determinar a melhor pontuação possível. Entretanto, esse tipo de lógica é desnecessário para a finalidade do modo *attract*.

Etapa 4

Prompt1:

Leia o código e o arquivo descricao.md contidos em @etapa3.

Agora, em @etapa4, use o mesmo código, porém agora adicione música e efeitos sonoros ao jogo, como os que já existem em tetris clássico de game boy. Adicione também efeitos visuais ao completar uma linha, ao subir de nível e ao dar game over por exemplo.

Em @etapa4 adicione um arquivo descricao.md que segue o mesmo estilo do que existe em @etapa3, porém no fim do arquivo detalhe o que foi adicionado, excluído e modificado.

OBS.: Deu overthinking

Prompt2:

Leia o código e o arquivo descricao.md contidos em @etapa3.

Agora, em @etapa4, use o mesmo código, porém agora adicione música e efeitos sonoros ao jogo, como os que já existem em tetris clássicos de game boy.

Em @etapa4 adicione um arquivo descricao.md que segue o mesmo estilo do que existe em @etapa3, porém no fim do arquivo detalhe o que foi adicionado, excluído e modificado.

Prompt3:

Leia o código e o arquivo descricao.md contidos em @etapa4.

Adicione efeitos visuais ao completar uma linha, ao subir de nível e ao dar game over.

Descreva as modificações feitas no arquivo [descricao.md](#).

Análise do Código, Fim da etapa 4

Nesta etapa, o agente apresentou uma solução surpreendente. Em vez de orientar apenas a inserção e reprodução de arquivos de áudio, ele utilizou uma API de áudio para recriar do zero a música e os efeitos sonoros do Tetris. Esse é um tipo de implementação que seria inviável para nós, considerando nossa ausência de conhecimento musical. Entretanto, com o auxílio do agente, foi possível realizar essa tarefa com apenas um prompt.

Conclusões e comentários

Após a realização de todas as etapas, observa-se que o agente escolhido foi capaz de concluir a tarefa com um grau considerável de competência. Mesmo quando erros eram identificados, ao serem apontados, o agente conseguia corrigi-los com facilidade.

Acreditamos que nossa falta de experiência com esse tipo de problema contribuiu para uma dependência excessiva do agente durante o desenvolvimento. Em alguns

momentos, por exemplo, solicitamos a implementação de funções que já haviam sido criadas anteriormente, algo que poderia ter sido evitado com uma análise mais cuidadosa do código. Além disso, quanto maior a complexidade do projeto, maior parecia ser a dificuldade do agente em realizar as implementações. Conforme o projeto evoluía, tornava-se necessário um conhecimento mais aprofundado sobre as tecnologias utilizadas e sobre desenvolvimento de jogos, tanto para orientar melhor o agente quanto para identificar bugs e trechos problemáticos do código.

Foi possível adquirir conhecimentos introdutórios de JavaScript e desenvolvimento de jogos, suficientes para compreender superficialmente o funcionamento do código. Entretanto, caso fosse necessário desenvolver um projeto semelhante sem o uso de agentes, a dificuldade seria significativamente maior. Embora viável, o tempo necessário seria muito superior ao gasto com o auxílio dessas ferramentas. A elaboração deste projeto demandou aproximadamente um dia inteiro; sem o uso de agentes de inteligência artificial, estimamos que a conclusão levaria pelo menos uma semana.

O código final apresenta legibilidade razoável, porém ainda é confuso e desorganizado, exigindo maior atenção para sua compreensão. Como toda a implementação foi realizada em um único arquivo e sem utilização de classes, a manutenção do código torna-se uma tarefa complexa. Conforme descrito ao longo do relatório, cada nova funcionalidade adicionada aumentava a desorganização do projeto, podendo chegar a um ponto em que a manutenção se tornaria inviável. Dessa forma, considera-se essencial realizar um processo de refatoração, tanto para melhorar a sustentabilidade quanto a segurança do código.

A compreensão do código obtida foi limitada em comparação ao que seria alcançado caso a implementação não tivesse dependido tão fortemente dos agentes. Alterações simples seriam possíveis, porém funcionalidades mais complexas provavelmente exigiriam novamente o auxílio dessas ferramentas. Além disso, apenas a análise do código não é suficiente para garantir que uma modificação tenha sido bem-sucedida, sendo necessário testar a aplicação para validar seu funcionamento.

Por fim, vale destacar que também foi realizado um experimento utilizando a estrutura HTML do jogo no GPT gratuito, solicitando a geração do arquivo JavaScript correspondente. Contudo, o resultado obtido foi significativamente inferior ao produzido pelo agente da Anthropic. A seguir, apresenta-se o código gerado pelo GPT gratuito:

